



Step 1: Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    roc_auc_score,
    RocCurveDisplay
)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline
```

Step 2: Load Data

```
In [2]: df = pd.read_csv("synthetic_fraud_dataset.csv")

df.head()
```

```
Out[2]:
```

	Transaction_ID	User_ID	Transaction_Amount	Transaction_Type	Timestamp
0	TXN_33553	USER_1834	39.79	POS	2023-08-14 19:30:00
1	TXN_9427	USER_7875	1.19	Bank Transfer	2023-06-07 04:01:00
2	TXN_199	USER_2734	28.96	Online	2023-06-20 15:25:00
3	TXN_12447	USER_2617	254.32	ATM Withdrawal	2023-12-07 00:31:00
4	TXN_39489	USER_2014	31.28	POS	2023-11-11 23:44:00

5 rows × 21 columns

Step 3: Basic Information

```
In [3]: df.info()  
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 50000 entries, 0 to 49999  
Data columns (total 21 columns):  
#   Column                                Non-Null Count  Dtype    
---  ---                                -  
0   Transaction_ID                        50000 non-null  object   
1   User_ID                             50000 non-null  object   
2   Transaction_Amount                   50000 non-null  float64  
3   Transaction_Type                     50000 non-null  object   
4   Timestamp                           50000 non-null  object   
5   Account_Balance                     50000 non-null  float64  
6   Device_Type                         50000 non-null  object   
7   Location                            50000 non-null  object   
8   Merchant_Category                   50000 non-null  object   
9   IP_Address_Flag                     50000 non-null  int64    
10  Previous_Fraudulent_Activity         50000 non-null  int64    
11  Daily_Transaction_Count              50000 non-null  int64    
12  Avg_Transaction_Amount_7d            50000 non-null  float64  
13  Failed_Transaction_Count_7d          50000 non-null  int64    
14  Card_Type                           50000 non-null  object   
15  Card_Age                            50000 non-null  int64    
16  Transaction_Distance                 50000 non-null  float64  
17  Authentication_Method                50000 non-null  object   
18  Risk_Score                          50000 non-null  float64  
19  Is_Weekend                          50000 non-null  int64    
20  Fraud_Label                          50000 non-null  int64    
dtypes: float64(5), int64(7), object(9)  
memory usage: 8.0+ MB
```

```
Out[3]:
```

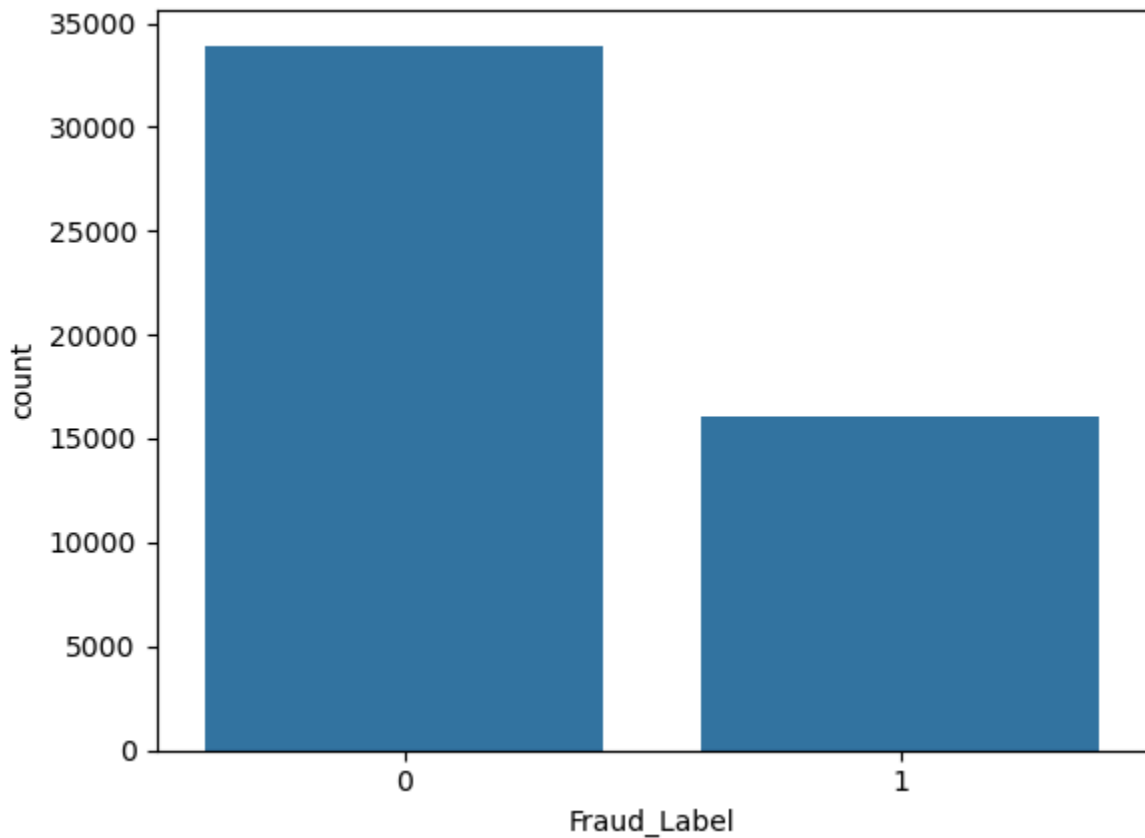
	Transaction_Amount	Account_Balance	IP_Address_Flag	Previous_Fraudul
count	50000.000000	50000.000000	50000.00000	5
mean	99.411012	50294.065981	0.05020	
std	98.687292	28760.458557	0.21836	
min	0.000000	500.480000	0.00000	
25%	28.677500	25355.995000	0.00000	
50%	69.660000	50384.430000	0.00000	
75%	138.852500	75115.135000	0.00000	
max	1174.140000	99998.310000	1.00000	

Step 4: Check Class Imbalance

```
In [4]: df['Fraud_Label'].value_counts()

sns.countplot(x='Fraud_Label', data=df)
plt.show()

df['Fraud_Label'].value_counts(normalize=True)*100
```



```
Out[4]: Fraud_Label
0      67.866
1      32.134
Name: proportion, dtype: float64
```

Step 5: Check Missing Values

```
In [5]: df.isnull().sum()
```

```
Out[5]: Transaction_ID      0
        User_ID            0
        Transaction_Amount  0
        Transaction_Type    0
        Timestamp          0
        Account_Balance    0
        Device_Type        0
        Location           0
        Merchant_Category  0
        IP_Address_Flag    0
        Previous_Fraudulent_Activity  0
        Daily_Transaction_Count  0
        Avg_Transaction_Amount_7d  0
        Failed_Transaction_Count_7d  0
        Card_Type          0
        Card_Age           0
        Transaction_Distance  0
        Authentication_Method  0
        Risk_Score         0
        Is_Weekend         0
        Fraud_Label        0
        dtype: int64
```

Step 6: Feature Selection

```
In [6]: X = df.drop(
        ['Fraud_Label',
         'Transaction_ID',
         'User_ID'],
        axis=1
    )

    y = df['Fraud_Label']
```

Step 7: Train-Test Split

```
In [7]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
```

Step 8: Identify Numerical and Categorical Columns

```
In [8]: cat_cols = X.select_dtypes(include='object').columns
        num_cols = X.select_dtypes(exclude='object').columns
```

Step 8: Preprocessing

```
In [9]: preprocessor = ColumnTransformer(  
        transformers=[  
            ('num', StandardScaler(), num_cols),  
            ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)  
        ]  
    )
```

Model 1: Logistic Regression Pipeline

```
In [10]: lr_pipeline = Pipeline([  
        ('preprocessor', preprocessor),  
        ('smote', SMOTE(random_state=42)),  
        ('classifier', LogisticRegression(  
            max_iter=1000,  
            random_state=42  
        ))  
    ])
```

```
In [11]: ## Train Logistic Regression  
lr_pipeline.fit(X_train, y_train)  
  
y_pred_lr = lr_pipeline.predict(X_test)  
y_prob_lr = lr_pipeline.predict_proba(X_test)[:,1]
```

```
In [12]: ## Evaluation  
print(classification_report(y_test,  
                            y_pred_lr))  
  
print("Precision:",  
      precision_score(y_test,  
                      y_pred_lr))  
  
print("Recall:",  
      recall_score(y_test,  
                   y_pred_lr))  
  
print("ROC-AUC:",  
      roc_auc_score(y_test,  
                    y_prob_lr))
```

	precision	recall	f1-score	support
0	0.87	0.81	0.84	6787
1	0.65	0.74	0.69	3213
accuracy			0.79	10000
macro avg	0.76	0.77	0.76	10000
weighted avg	0.80	0.79	0.79	10000

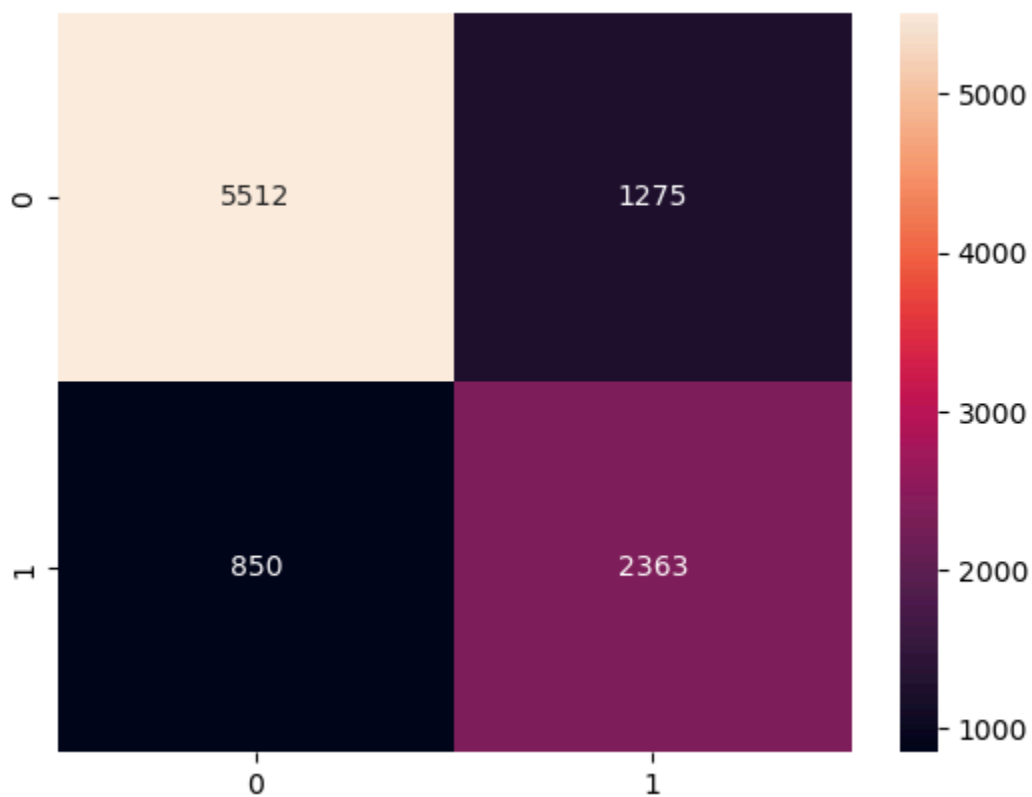
Precision: 0.6495327102803738

Recall: 0.7354497354497355

ROC-AUC: 0.8896121092707993

```
In [13]: ## Confusion Matrix
cm = confusion_matrix(
    y_test,
    y_pred_lr
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d'
)
plt.show()
```



Model 2: Random Forest Pipeline

```
In [14]: rf_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(
        random_state=42
    ))
])
```

```
In [15]: ## Train Random Forest
rf_pipeline.fit(X_train, y_train)

y_pred_rf = rf_pipeline.predict(X_test)
y_prob_rf = rf_pipeline.predict_proba(X_test)[:,1]
```

```
In [16]: ## Evaluation
print(classification_report(
    y_test,
    y_pred_rf
))

print("Precision:",
    precision_score(
        y_test,
        y_pred_rf
    ))

print("Recall:",
    recall_score(
        y_test,
        y_pred_rf
    ))

print("ROC-AUC:",
    roc_auc_score(
        y_test,
        y_prob_rf
    ))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6787
1	1.00	0.99	1.00	3213
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

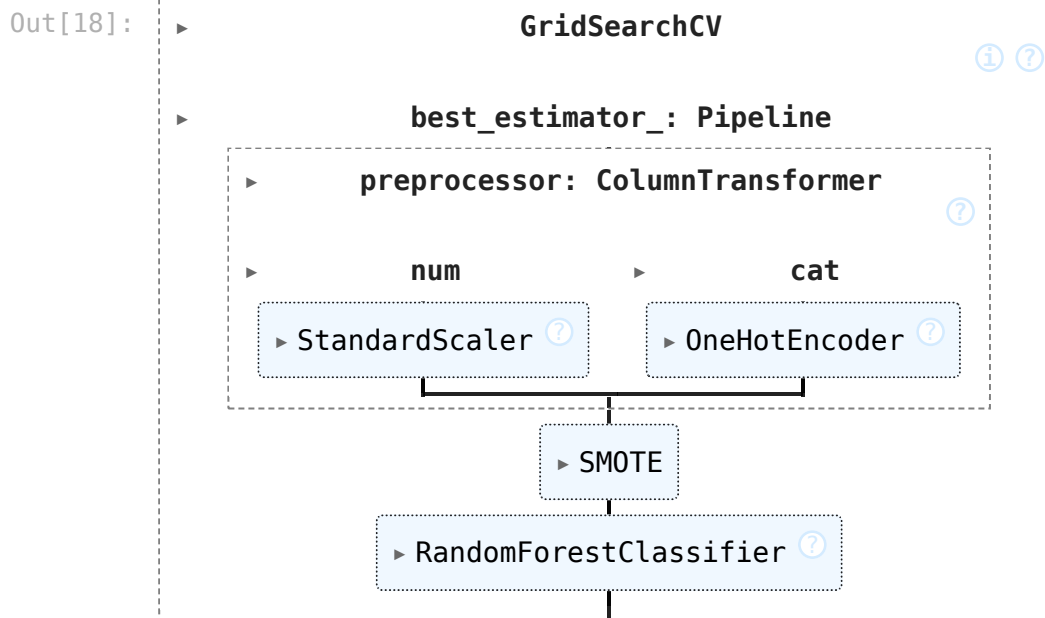
Precision: 0.9996862252902416
Recall: 0.9915966386554622
ROC-AUC: 0.9999961250318767

Hyperparameter Tuning (GridSearchCV)

```
In [18]: param_grid = {
    'classifier__n_estimators':[100],
    'classifier__max_depth':[10,20],
    'smote__k_neighbors':[5]
}

grid = GridSearchCV(
    rf_pipeline,
    param_grid,
    cv=3,
    scoring='roc_auc',
    n_jobs=-1
)

grid.fit(X_train,
        y_train)
```



```
In [19]: ## Best Model
print(grid.best_params_)
print(grid.best_score_)

{'classifier__max_depth': 20, 'classifier__n_estimators': 100, 'smote__k_neighbors': 5}
0.9997127084062081
```

```
In [20]: ## Final Predictions
best_model = grid.best_estimator_

pred = best_model.predict(X_test)
```



```

prob = best_model.predict_proba(X_test)[:,1]

print(classification_report(
    y_test,
    pred
))

print("ROC-AUC:",
      roc_auc_score(
        y_test,
        prob
      ))

```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	6787
1	1.00	0.88	0.94	3213
accuracy			0.96	10000
macro avg	0.97	0.94	0.95	10000
weighted avg	0.96	0.96	0.96	10000

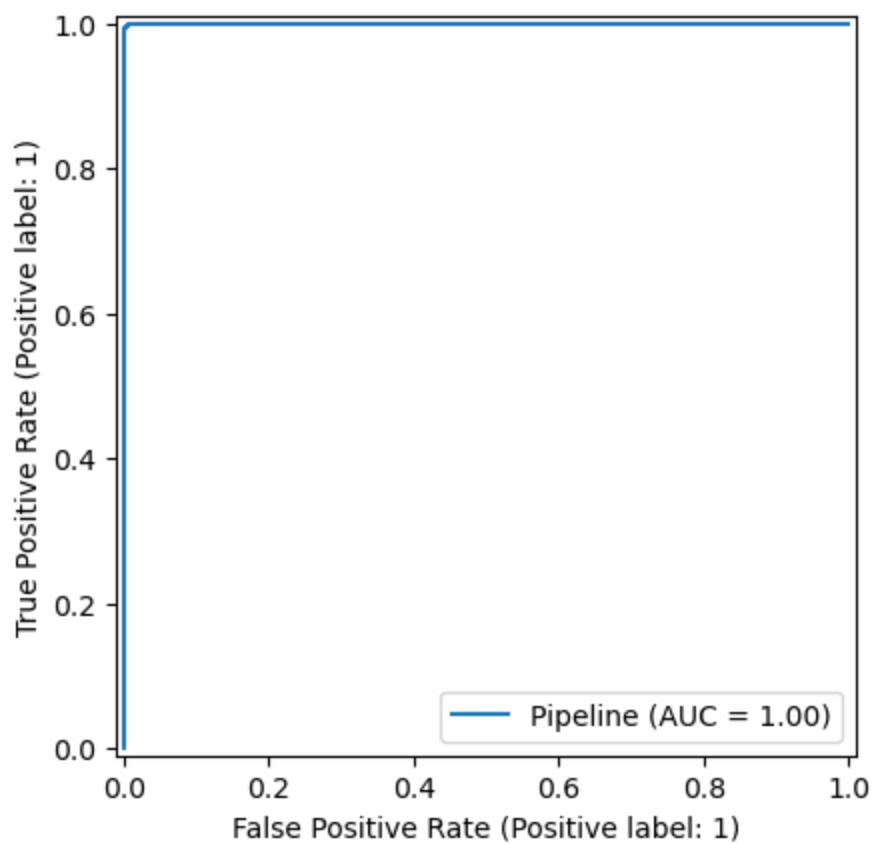
ROC-AUC: 0.9999385049437486

```

In [21]: ## ROC Curve
RocCurveDisplay.from_estimator(
    best_model,
    X_test,
    y_test
)

plt.show()

```



```
In [22]: ## Feature Importance (Random Forest)  
importances = best_model.named_steps[  
    'classifier'  
].feature_importances_
```

```
In [ ]:
```